

PDD

Sistema Integrado de Agendamento Escolar

Laboratórios, Biblioteca, Espaço Maker e Agenda Acadêmica

ETEC Antônio Furlan | Hackathon de 4 horas

Propósito do documento

Servir como fonte única de verdade para a equipe e como contexto de implementação para o agente de IA no Antigravity, utilizando Claude Opus. O foco é entregar um MVP demonstrável, consistente e executável dentro do tempo disponível.

Versão 1.0 • 15 de agosto de 2026

Equipe de desenvolvimento: 5 integrantes

1. Resumo executivo

O projeto é uma aplicação web única para centralizar a reserva de espaços escolares e a divulgação de compromissos acadêmicos. O MVP integra três tipos de espaço físico, laboratórios de informática, biblioteca e Espaço Maker, além de uma agenda de provas, trabalhos, visitas e prazos por turma.

Decisão de escopo

Em quatro horas, a prioridade é um fluxo ponta a ponta funcional: consultar agenda, criar/editar/cancelar reservas sem conflito, administrar espaços e publicar eventos acadêmicos. Autenticação real, notificações e recorrência avançada ficam simplificadas ou como extensão.

2. Contexto e problema

- Reservas atuais são manuais ou descentralizadas, causando choques de horário e pouca visibilidade.
- Professores precisam localizar rapidamente um laboratório compatível com equipamentos e softwares da aula.
- Alunos enfrentam dificuldade para entender regras, capacidade, horários e autorização da biblioteca e do Espaço Maker.
- Prazos acadêmicos ficam dispersos; representantes precisam publicar e alunos precisam consultar uma agenda comum.
- Cancelamentos tardios deixam espaços ociosos e impedem o reaproveitamento do horário.

3. Objetivo do produto

Permitir que a comunidade escolar visualize, cadastre e gerencie reservas e eventos em uma interface simples, responsiva e centralizada, reduzindo conflitos e tornando a disponibilidade transparente.

3.1 Indicadores de sucesso da demonstração

- Uma reserva válida é criada em até 60 segundos.
- O sistema bloqueia duas reservas ativas para o mesmo espaço em horários sobrepostos.
- Um usuário visualiza horários livres e ocupados por dia ou semana.
- Um administrador consegue cadastrar e editar espaços e seus recursos.
- Um representante publica um evento acadêmico e um aluno consegue visualizá-lo.
- Cancelamento libera imediatamente o horário para nova reserva.

4. Escopo do MVP

Incluído no MVP	Fora do MVP ou extensão
Dashboard com atalhos e resumo do dia	Notificações por e-mail, push ou WhatsApp
CRUD de espaços, reservas e eventos acadêmicos	Integração com sistemas oficiais da escola
Agenda diária e semanal; visão mensal simples se houver tempo	Sincronização com Google/Outlook Calendar
Filtros por tipo de espaço, data e turma	SSO institucional e recuperação de senha
Detecção de conflito de horários	Auditoria completa e relatórios avançados
Recorrência simples com data final	Exceções complexas de séries recorrentes

Incluído no MVP	Fora do MVP ou extensão
Perfis simulados e permissões na interface/API	Gestão real de identidade e segurança de produção
Layout responsivo para notebook e smartphone	Aplicativo móvel nativo

5. Usuários e permissões

Perfil	Pode visualizar	Pode criar/alterar
Aluno	Agenda acadêmica, espaços, regras e disponibilidade	Solicitações para biblioteca/Maker, se habilitado; não edita agenda acadêmica
Representante de sala	Tudo do aluno e calendário da turma	Eventos acadêmicos da própria turma
Professor	Espaços, recursos e todas as reservas relevantes	Reservas pontuais e recorrentes de laboratórios; cancelar as próprias
Bibliotecário/Funcionário	Solicitações e agenda de biblioteca/Maker	Aprovar, rejeitar, editar e cancelar utilizações
Gestor/Administrador	Todos os módulos	CRUD de espaços, usuários simulados, reservas e eventos
Técnico de TI	Laboratórios, reservas e recursos	Atualizar equipamentos e softwares dos laboratórios

6. Requisitos funcionais

ID	Requisito	Critério resumido
RF01	Alternar perfil de demonstração	Selecionar um usuário/perfil sem login real para demonstrar permissões.
RF02	Listar espaços	Exibir nome, tipo, capacidade, status, localização, equipamentos, softwares e regras.
RF03	Gerenciar espaços	Administrador cria, consulta, edita, ativa/inativa e exclui logicamente um espaço.
RF04	Consultar agenda	Visualizar reservas por dia e semana; mês é desejável.
RF05	Filtrar disponibilidade	Filtrar por tipo, data, horário, capacidade, equipamento ou software.
RF06	Criar reserva	Registrar espaço, solicitante, data, início, fim, finalidade e quantidade de pessoas.
RF07	Validar conflito	Impedir sobreposição de reservas ativas no mesmo espaço.
RF08	Gerenciar reserva	Editar, cancelar e consultar detalhes, respeitando permissões.
RF09	Aprovação	Reservas de biblioteca/Maker podem iniciar como PENDENTE e ser aprovadas/rejeitadas.
RF10	Recorrência simples	Criar ocorrências diárias ou semanais até uma data final.
RF11	Cancelar ocorrência	Cancelar uma ocorrência sem apagar as demais, se implementada a recorrência.

ID	Requisito	Critério resumido
RF12	Agenda acadêmica	Representante/gestor cria, edita e exclui evento associado a uma turma.
RF13	Consulta do aluno	Aluno vê eventos da própria turma, sem edição.
RF14	Feedback imediato	Mostrar mensagens claras de sucesso, validação, conflito e erro.

7. Regras de negócio

ID	Regra
RN01	Não pode existir sobreposição entre reservas com status PENDENTE ou APROVADA para o mesmo espaço.
RN02	Uma reserva deve ter início anterior ao fim e ocorrer dentro do horário operacional cadastrado.
RN03	A quantidade de participantes não pode superar a capacidade do espaço.
RN04	Reservas CANCELADAS ou REJEITADAS não bloqueiam disponibilidade.
RN05	Laboratórios podem ser reservados diretamente por professores/gestores; biblioteca e Maker podem exigir aprovação.
RN06	Somente o criador ou perfis administrativos podem editar/cancelar uma reserva.
RN07	Somente representante da turma ou gestor pode alterar os eventos acadêmicos daquela turma.
RN08	Exclusão de espaço deve ser lógica quando houver histórico de reservas.
RN09	Recorrência gera ocorrências independentes ligadas por um identificador de série.
RN10	Datas e horários são armazenados no fuso America/Sao_Paulo.

8. Fluxos principais

8.1 Reservar um espaço

1. Usuário abre Espaços ou Agenda.
2. Aplica filtros e escolhe um espaço.
3. Seleciona data, início e fim.
4. Informa finalidade e participantes.
5. Sistema valida dados, capacidade e conflito.
6. Sistema grava como APROVADA ou PENDENTE conforme tipo/perfil.
7. Agenda é atualizada e exibe confirmação.

8.2 Publicar compromisso acadêmico

8. Representante seleciona sua turma.
9. Informa título, tipo, disciplina, data, horário opcional e descrição.
10. Sistema valida permissão e grava o evento.
11. Alunos da turma visualizam o compromisso no calendário.

9. Telas do MVP

Tela	Conteúdo e ações
1. Entrada/Demo	Logo/nome, seletor de usuário e perfil, botão Entrar no sistema.
2. Dashboard	Reservas de hoje, próximos eventos, espaços disponíveis e atalhos.
3. Agenda de espaços	Calendário diário/semanal, filtros, blocos por status e botão Nova reserva.
4. Espaços	Cards/lista, recursos, capacidade, regras, disponibilidade e CRUD administrativo.
5. Formulário de reserva	Espaço, data, início/fim, finalidade, participantes e recorrência simples.
6. Minhas reservas	Lista com status, edição e cancelamento.
7. Agenda acadêmica	Calendário/lista por turma e formulário de evento conforme permissão.
8. Administração	CRUD de espaços e dados auxiliares; usuários apenas simulados no MVP.

10. Arquitetura proposta

Stack recomendada para velocidade

Frontend React + Vite + TypeScript; backend Node.js + Express + TypeScript; banco SQLite; ORM Prisma; UI com Tailwind CSS; calendário simplificado com componentes próprios ou biblioteca já disponível. Manter frontend e backend em pastas separadas no mesmo repositório.

- frontend/: páginas, componentes, serviços HTTP, tipos e validações de formulário.
- backend/: rotas, controllers, services, validações, Prisma e seed.
- Banco local SQLite para evitar configuração externa durante o hackathon.
- API REST JSON com validação no backend e mensagens de erro consistentes.
- Seed obrigatório com usuários, turmas, espaços, reservas e eventos para a demo.

10.1 Estrutura sugerida do repositório

```

/frontend
  /src/components /pages /services /types
/backend
  /src/routes /controllers /services /middlewares
  /prisma/schema.prisma /prisma/seed.ts
README.md .env.example

```

11. Modelo de dados

Entidade	Campos essenciais
User	id, name, email, role, classId?, active
ClassGroup	id, name, course, grade
Space	id, name, type, location, capacity, description, rules, requiresApproval, active
Resource	id, spaceId, category, name, version?, quantity?

Entidade	Campos essenciais
Reservation	id, spaceId, userId, title, purpose, startAt, endAt, participants, status, recurrenceGroupId?, createdAt
AcademicEvent	id, classId, createdById, title, type, subject?, description?, startAt, endAt?
RecurrenceGroup	id, frequency, interval, untilDate

11.1 Enumerações

- UserRole: STUDENT, CLASS_REP, TEACHER, STAFF, LIBRARIAN, IT_TECH, ADMIN.
- SpaceType: LAB, LIBRARY, MAKER.
- ReservationStatus: PENDING, APPROVED, REJECTED, CANCELLED.
- AcademicEventType: EXAM, ASSIGNMENT, DEADLINE, VISIT, ACTIVITY, OTHER.

12. Contrato mínimo da API

Método	Rota	Finalidade
GET	/api/spaces	Listar e filtrar espaços
POST	/api/spaces	Criar espaço
PUT	/api/spaces/:id	Editar espaço
DELETE	/api/spaces/:id	Inativar espaço
GET	/api/reservations	Listar por período, espaço, usuário ou status
POST	/api/reservations	Criar, validar e opcionalmente gerar recorrências
PUT	/api/reservations/:id	Editar com nova validação de conflito
PATCH	/api/reservations/:id/status	Aprovar, rejeitar ou cancelar
GET	/api/events	Listar eventos por turma/período
POST	/api/events	Criar evento acadêmico
PUT	/api/events/:id	Editar evento
DELETE	/api/events/:id	Excluir evento
GET	/api/users	Listar usuários do modo demonstração
GET	/api/classes	Listar turmas

13. Requisitos não funcionais

- Responsivo a partir de 360 px e utilizável em notebook.
- Operações principais devem responder em até 2 segundos no ambiente local.
- Contraste e foco de teclado visíveis; campos com labels e mensagens de erro.
- Código TypeScript organizado, sem segredos no repositório e com .env.example.
- Validação duplicada: experiência no frontend e integridade no backend.

- README com instalação, migração, seed e execução em poucos comandos.
- Erros da API no padrão: { error, message, details? }.

14. Dados de demonstração

- 3 laboratórios com capacidades e softwares distintos.
- 1 biblioteca e 1 Espaço Maker, ambos com regras e capacidade.
- Ao menos 1 usuário de cada perfil principal e 2 turmas.
- Reservas aprovadas, pendentes e canceladas em dias próximos.
- Prova, trabalho, visita e prazo acadêmico em uma turma.
- Um cenário proposital para demonstrar bloqueio de conflito.

15. Plano de execução em 4 horas

Janela	Foco	Saída esperada
00:00-00:20	Alinhar PDD, stack, repositório e distribuição	Escopo congelado; projeto inicial rodando
00:20-01:10	Banco, schema, seed e API base	CRUD de espaços e reservas; dados de demo
01:10-02:10	Frontend base e navegação	Dashboard, espaços, agenda e formulários
02:10-03:00	Integração e regras críticas	Conflito, status, eventos acadêmicos e permissões
03:00-03:35	Polimento e testes	Responsividade, mensagens e correções
03:35-04:00	Congelamento e apresentação	README, roteiro de demo e versão estável

15.1 Divisão sugerida para 5 pessoas

Pessoa	Responsabilidade primária
1	Liderança técnica, integração, Git e suporte ao agente de IA.
2	Banco, Prisma, seed e regras de conflito.
3	API REST, validações e permissões.
4	Frontend: agenda, espaços e reservas.
5	Frontend: dashboard, agenda acadêmica, UX, testes e apresentação.

16. Instruções para o agente no AntigraVity

Modo de trabalho

Trate este PDD como contrato do MVP. Não acrescente funcionalidades fora do escopo antes de concluir os critérios de aceite. Faça mudanças pequenas, executáveis e verificáveis. Após cada etapa, rode o projeto e os testes pertinentes.

- Primeiro, criar um plano curto e a árvore de arquivos; depois implementar sem pedir confirmação entre cada arquivo.
- Usar TypeScript em frontend e backend, nomes em inglês no código e textos da interface em português do Brasil.
- Não inventar integrações externas. Usar SQLite e dados seed locais.
- Implementar a validação de conflito no backend como regra obrigatória.
- Fornecer comandos exatos para instalar, migrar, popular e executar.
- Priorizar funcionalidade e estabilidade. Evitar abstrações excessivas e dependências desnecessárias.
- Quando uma decisão não estiver definida, escolher a alternativa mais simples compatível com este documento e registrar no README.
- Não substituir arquivos funcionais inteiros sem necessidade; preservar o trabalho paralelo da equipe.

17. Critérios de aceite do MVP

- ☐ Projeto inicializa localmente seguindo o README.
- ☐ Seed cria espaços, usuários, turmas, reservas e eventos.
- ☐ É possível cadastrar, editar e inativar um espaço.
- ☐ É possível criar uma reserva válida e vê-la na agenda.
- ☐ Reserva sobreposta ao mesmo espaço é recusada com mensagem clara.
- ☐ Cancelamento libera o horário.
- ☐ Biblioteca/Maker pode gerar reserva pendente e funcionário pode aprovar.
- ☐ Representante cria evento para sua turma e aluno apenas visualiza.
- ☐ Interface funciona em tela móvel e desktop sem quebra crítica.
- ☐ Demonstração completa pode ser executada em no máximo 5 minutos.

18. Riscos e mitigação

Risco	Mitigação
Escopo exceder quatro horas	Congelar MVP; recorrência avançada e mês completo são extensões.
Conflitos de Git	Responsáveis por áreas; branches curtas; integrar a cada 30-45 minutos.
Agente alterar trabalho alheio	Prompts por tarefa, contexto de arquivos e commits frequentes.
Calendário consumir tempo	Começar com agenda diária/semanal simples; biblioteca externa apenas se estável.
Autenticação travar a entrega	Usar seletor de perfil mockado e middleware por header no MVP.
Falha na apresentação	Seed determinístico, roteiro ensaiado e versão estável congelada 25 minutos antes.

19. Roteiro de demonstração

12. Entrar como professor e filtrar laboratório por software.
13. Criar reserva válida e mostrar confirmação.
14. Tentar criar reserva sobreposta e mostrar bloqueio.
15. Cancelar a reserva e comprovar que o horário foi liberado.
16. Entrar como aluno, solicitar Biblioteca ou Maker e mostrar status pendente.
17. Entrar como funcionário e aprovar a solicitação.
18. Entrar como representante, publicar uma prova e visualizar como aluno.
19. Entrar como administrador e editar a capacidade ou os recursos de um espaço.

20. Premissas registradas

- O sistema será demonstrado localmente e não precisa ser publicado para cumprir o MVP.
- Os perfis serão simulados; não haverá senha nem dados pessoais reais.
- Os horários operacionais podem ser configurados por espaço ou definidos no seed.
- Aprovação é obrigatória apenas quando requiresApproval estiver ativo.
- A visão mensal é desejável, mas não bloqueia a entrega se a semanal estiver funcional.
- Este documento consolida os quatro documentos de descoberta fornecidos pela equipe.

21. Referências de origem

Documentos fornecidos pela equipe: “Agendamento - UDIAD”; “Documento sem título (7)”; “Tema Sistema de Reserva de Laboratórios de Informática para”; e “Desenvolvimento de uma Solução Digital Centrada no Usuário”. Consulta complementar: página oficial do Google Antigravity, usada apenas para contextualizar o ambiente agent-first; as decisões técnicas deste PDD são recomendações específicas para o hackathon.